

Revisiting Neural Retrieval on Accelerators

Jiaqi Zhai
jiaqiz@meta.com
Meta Platforms, Inc.
Menlo Park, CA, USA

Zhaojie Gong
zhaojie@meta.com
Meta Platforms, Inc.
Menlo Park, CA, USA

Yueming Wang
yuemingw@meta.com
Meta Platforms, Inc.
Menlo Park, CA, USA

Xiao Sun
sunx@meta.com
Meta Platforms, Inc.
Menlo Park, CA, USA

Zheng Yan
zyan@meta.com
Meta Platforms, Inc.
Menlo Park, CA, USA

Fu Li
leaf123@meta.com
Meta Platforms, Inc.
Menlo Park, CA, USA

Xing Liu
xingl@meta.com
Meta Platforms, Inc.
Menlo Park, CA, USA

ABSTRACT

Retrieval finds a small number of relevant candidates from a large corpus for information retrieval and recommendation applications. A key component of retrieval is to model (user, item) similarity, which is commonly represented as the dot product of two learned embeddings. This formulation permits efficient inference, commonly known as **Maximum Inner Product Search (MIPS)**. Despite its popularity, dot products cannot capture complex user-item interactions, which are multifaceted and likely high rank. We hence examine non-dot-product retrieval settings on accelerators, and propose *mixture of logits* (MoL), which models (user, item) similarity as an adaptive composition of elementary similarity functions. This new formulation is expressive, capable of modeling high rank (user, item) interactions, and further generalizes to the long tail. When combined with a hierarchical retrieval strategy, *h-indexer*, we are able to scale up MoL to 100M corpus on a single GPU with latency comparable to MIPS baselines. On public datasets, our approach leads to uplifts of up to 77.3% in hit rate (HR). Experiments on a large recommendation surface at Meta showed strong metric gains and reduced popularity bias, validating the proposed approach’s performance and improved generalization.

CCS CONCEPTS

• **Information systems** → **Learning to rank; Recommender systems**; • **Computing methodologies** → **Machine learning algorithms**.

KEYWORDS

information retrieval, recommender systems, candidate generation, non-MIPS retrieval, hierarchical retrieval, nearest neighbor search

ACM Reference Format:

Jiaqi Zhai, Zhaojie Gong, Yueming Wang, Xiao Sun, Zheng Yan, Fu Li, and Xing Liu. 2023. Revisiting Neural Retrieval on Accelerators. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD ’23)*, August 6–10, 2023, Long Beach, CA, USA. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3580305.3599897>

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

KDD ’23, August 6–10, 2023, Long Beach, CA, USA

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0103-0/23/08.

<https://doi.org/10.1145/3580305.3599897>

1 INTRODUCTION

Retrieval, also known as candidate generation, is the process of selecting a small number, typically 10-1000, of relevant candidates from a large corpus consisting of millions to billions of items. Retrieval plays a crucial role in large scale information retrieval and recommendation systems as the first stage of the funnel. Retrieval must capture (user, item) similarity well by being highly contextualized and personalized; for instance, an individual may want to read important economic news on weekday mornings, but watch lighthearted comedies in the evenings.

In the deep learning era, (user, item) similarity in retrieval is commonly formulated as the dot product of two embeddings, each parameterized by a deep neural network. This setting is widely adopted throughout industry (e.g., Google [3, 45], Meta [13], Alibaba [26, 48], etc.). The dot product setting empirically works well for millions of items [3], and is computationally efficient; for instance, the inference problem in this setup can be formulated as Maximum Inner Product Search (MIPS). MIPS has been extensively studied, and permits various optimizations such as locality sensitive hashing and product quantization [10, 33].

The relationship between users and items in real world, however, demonstrates a significant level of complexity, and may not be approximated well by dot products. There are two evidences. First, dot product-based models produce low-rank recommendations. Consider the log probability of showing item x to user u in the inner product setting, $\ln p(x|u) = \langle f_\theta(u), g_\theta(x) \rangle - \mathbb{Z}_u$, where $f_\theta(u), g_\theta(x) \in \mathbb{R}^d$. The $\ln p(x|u)$ matrix hence has a rank of at most $d + 1$. Commonly used d values are up to 100s-200s [3, 18, 25], which is far less than the rank of actual datasets. As shown in Table 1, even for small benchmark datasets, dot-product based 256-d models can only explain up to 7.11%-41.26% of the variance. Second, highly expressive neural networks (e.g., high order feature interactions [1, 9, 41, 43]; sequential encoders such as RNNs and

Table 1: Rank analysis of $\ln p(x|u)$ for various public datasets.

	# of users	# of items	Explained variance (SVD)		
			$d = 64$	$d = 256$	$d = 1024$
ML-1M	6,040	3,649	0.1744	0.4126	0.8221
ML-20M	138,493	24,186	0.1373	0.3304	0.6452
Beauty	40,226	54,243	0.0496	0.1212	0.2654
Games	29,341	23,310	0.1100	0.2355	0.4609
Books	694,897	674,044	0.0336	0.0711	0.1351

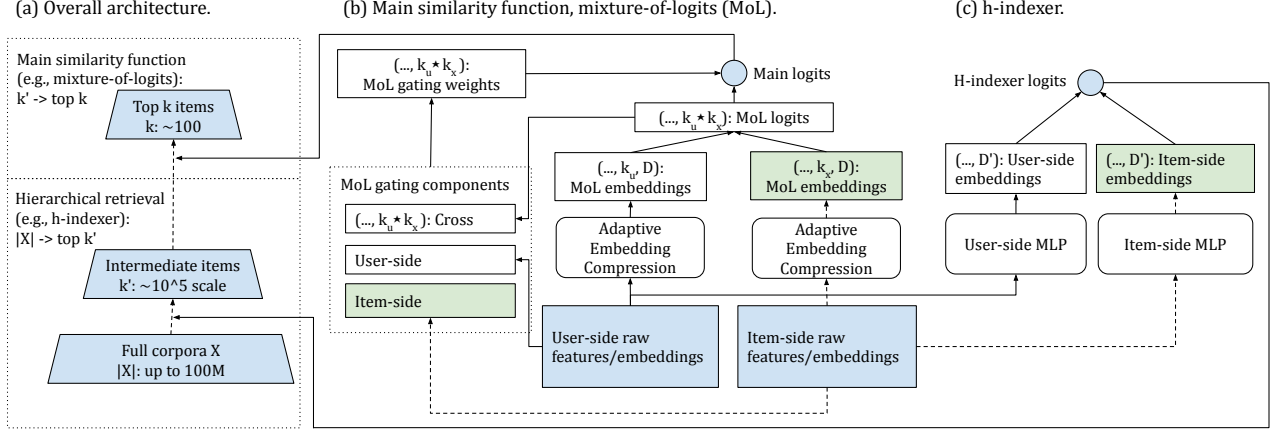


Figure 1: Overview of the proposed architecture. Green boxes indicate tensors that are cachable for inference.

Attention [36, 38, 49]) have been the go-to choice for ranking models. A more expressive similarity function, therefore, would better optimize metrics such as HR@10 for the head parts of the corpus.

Despite the above, leveraging expressive similarity functions in retrieval has remained an open ended problem due to (a) the difficulty of training models that generalize, (b) their high inference costs. It’s not sufficient to port an arbitrary neural architecture to retrieval; Rendel et al. [32] have recently shown that with properly tuned hyper parameters, a learned dot product can substantially outperform multilayer perceptron (MLP)-based learned similarity functions. It’s also hard for MLPs to approximate dot products despite MLPs being universal function approximators [12, 32]. Computational cost is another major blocker; modern information retrieval applications may need to handle corpus with up to billions of items [6, 39]. We lack efficient alternatives to MIPS at this scale. One line of work [50–52] proposes to learn explicit tree structures for the items and reduce top- k similarity to beam search at inference time, which enabled the first successful non-MIPS setup in production. The applications of these approaches have been limited: first, it is generally harder to learn explicit structures — for instance, prior knowledge are often needed to properly initialize the tree [50, 51], but the tree then needs to change dynamically as the corpus evolves; second, when binary tree structures are used [50, 51], inference latency can be high as beam search requires 22 non-parallelizable steps just to traverse a corpus with 4M items.

To address the above problems, we propose modeling techniques and infra optimizations that enable complex interactions of (user, item) pairs to be modeled at the retrieval stage while supporting efficient inference. We consider the accelerator setting in particular, given that the enablement and proliferation of accelerators like GPUs and TPUs [17] can significantly mitigate the training and serving cost of complex similarity functions.

Our first contribution is to propose a specific class of high rank similarity functions, *mixture of logits* (MoL), in Section 3. MoL is designed with efficient training and inference on accelerators in mind. It not only significantly outperforms dot-product based baselines in hit rate, but also better generalizes to torso and long tail where there are limited training examples by utilizing gating designs and component-level hypersphere embeddings.

Our second contribution is to enable efficient training and inference with expressive similarity functions (e.g., MoL), on modern accelerators (Section 4). We propose a hierarchical retrieval strategy, and in particular, an accelerator-friendly algorithm for very large corpus called *h-indexer*. Combined with other optimizations like quantization and kernel fusions, h-indexer enables us to scale up retrieval to hundreds of millions of items with latency comparable to MIPS setups. It’s worth remarking that, as highlighted in Figure 2, the serving cost, e.g., latency and throughput — on GPUs in this work — does *not* necessarily increase with compute thanks to the significantly higher GPU utilization of our model. Our hierarchical design, unlike dot-product, can also harness more compute with constant GPU memory budget, which is critical given limited memory capacity in production.

Our final contribution is to conduct extensive experiments on public and industrial datasets (Section 5) with MoL and h-indexer. Our proposed method achieves up to 77.3% gain in HR on standard benchmark datasets like MovieLens and Amazon Reviews, and up to +27.6% gain at HR@50 over strong baselines on one of the largest recommendation system deployments at Meta. The strong performance of MoL validates our hypothesis that expressive similarity functions are needed to capture dynamics of real-world datasets at retrieval stage. The proposed method also reduces Matthew effect, showing that MoL indeed generalizes well over the entire corpus.

We finally compare the proposed approach with related works in Section 6 and conclude in Section 7.

2 OVERVIEW

2.1 The Retrieval Problem Setting

In a standard retrieval setup, we are given a user u (or more generally, context c) and would like to predict the item x ’s that is most positively associated with the given user, where x ’s are selected out of a corpora $\mathbb{X}$. The user u can be characterized by various features, like categorical features (e.g., user id, demographics, accounts that the user follows, etc.), float features (e.g., various counters), etc.

Regardless of the features used, we aim to learn a function ϕ parameterized by θ that encodes similarity between user u and item x as $\phi_\theta(u, x)$. For a given ϕ , the goal is to learn θ such that we order

$\phi_\theta(u, x)$'s for x 's that are positively associated with the user ahead of other $\phi_\theta(u, x')$'s for $x' \in \mathbb{X}$.

It's useful to formulate the similarity between user u and item x as a probability distribution $p(x|u)$ [3, 48]. We do so by considering $\phi_\theta(x, u)$ as unnormalized logits and pass them through softmax:

$$p(x|u) = \frac{e^{\phi_\theta(x, u)}}{\sum_{x' \in \mathbb{X}} e^{\phi_\theta(x', u)}} \quad (1)$$

Specific to information retrieval and recommendation setting, the size of $\mathbb{X}$ can be very large, potentially in the range of millions to billions for practical problem settings. Dot products (two tower, dual encoder setups, etc.) are hence commonly used. In this setup, we learn user and item representations as two d -dimensional embeddings, $f_\theta(u) \in \mathbb{R}^d$ and $g_\theta(x) \in \mathbb{R}^d$. We recommend item x to user u with probability proportional to their inner products, $\phi_\theta(x, u) \sim \langle f_\theta(u), g_\theta(x) \rangle$; recall that we obtain normalized probability distribution with softmax, hence

$$\ln p(x|u) = \langle f_\theta(u), g_\theta(x) \rangle - Z_u \quad (2)$$

where Z_u is the partition function.

2.2 Architecture

Figure 1 shows our main architecture used in the rest of the paper. We highlight the main components of this architecture below.

Overall flow. Our design decomposes retrieval into multiple stages. These stages run in a cascading fashion to produce the final top k candidates. One example is illustrated in Figure 1(a), where an accelerator friendly algorithm, *h-indexer*, is used to find a large number of candidates ($k' = 10^5$) out of a 100M corpus, and then a complex similarity function (e.g., *mixture-of-logits*) is used to find the final top k (e.g., 100) candidates. We show in Section 5.2.1 that this design significantly improves throughput without degrading recall for suitably chosen values of k' .

Main similarity function. The hierarchical retrieval design enables complex neural networks to be used when modeling (user, item) similarities. In Section 3, we discuss one such instance of similarity functions, *mixture of logits*, that significantly outperforms dot products. When designing this architecture, we aggressively make intermediate tensors available for caching (green boxes in Fig. 1). For instance, we can cache item-side gating weights and combine them cheaply with non-cacheable weights at inference time.

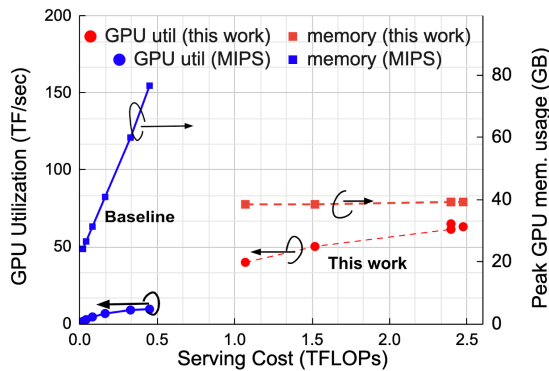


Figure 2: Infra efficiency in production: GPU utilization and peak memory scaling with serving FLOPs.

h-indexer. We found that a simple, but highly optimized dot product combined with specialized top- k algorithm works well for up to 100M corpus. This stage is co-trained with the main similarity function. We discuss this design in details in Section 4.1.

3 MIXTURE OF LOGITS: AN ACCELERATOR-AWARE MODEL DESIGN

In this section, we propose a new high rank similarity function, *mixture of logits* (MoL). MoL is designed for neural retrieval settings on accelerators. The basic idea in MoL is to parameterize $\phi_\theta(u, x)$ as an *adaptive* mixture of more elementary logits,

$$\phi_{MoL}(x, u) = \sum_k \pi_{k, \theta}(x, u) \delta_{k, \theta}(x, u) \quad (3)$$

where $\pi_{k, \theta}(x, u)$ are adaptive gating weights and $\delta_{k, \theta}(x, u)$ are elementary logits.

MoL achieves high rank in two ways. First, since $\delta_{k, \theta}(x, u)$ in Equation 3 can be parameterized by any form of neural networks, $\phi_\theta(x, u)$ has arbitrarily high rank. Second, as $\pi_{k, \theta}(x, u)$ takes x and u as input, we can create a high rank matrix $\ln p(x|u)$ by combining low rank similarity functions, e.g., dot products, in a cost-efficient fashion. For instance, using k groups of dot products (of potentially different embedding dimensions), we can derive

$$\phi_{MoL_{dot\ products}}(x, u) = \sum_k \pi_{k, \theta}(x, u) \langle f_{k, \theta}(u), g_{k, \theta}(x) \rangle \quad (4)$$

Although the elementary logits $\langle f_{k, \theta}(u), g_{k, \theta}(x) \rangle$ themselves have ranks of at most d (inner products of d -dimensional embeddings), Equation 4 can define an arbitrarily high rank probability distribution, as long as $\pi_{k, \theta}(x, u)$ depends on *both* item x and user u . Otherwise, suppose $\pi_{k, \theta}(x, u)$ only depends on u , we have

$$\sum_k \pi_{k, \theta}(u) \langle f_{k, \theta}(u), g_{k, \theta}(x) \rangle = \sum_k \langle \pi_{k, \theta}(u) f_{k, \theta}(u), g_{k, \theta}(x) \rangle \quad (5)$$

which defines $\phi_{MoL}(x, u)$ of rank at most kd when all embeddings are of dimensionality d . Similar argument holds on the item side.

Empirically this formulation does enable us to achieve high rank, as shown in Section 5.1.2.

3.1 Embedding Sharing and Adaptive Embedding Compression

Implementing Equation 4 directly requires k matrix multiplication (*mm*) / batched matrix multiplication (*bmm*) operations. We can accelerate training and inference as follows: (a) make embedding dimensions used across different mixture components the same, and (b) reuse embeddings across these groups and enable k to be different on the user and on the item side. Specifically, we consider k_u embeddings of dimensionality d on the user side and k_x embeddings of the same dimension on the item side. We can then take dot products between all k_u -by- k_x pairs of embeddings:

$$\phi_{MoL_{dot\ products'}}(x, u) = \sum_{k_u, k_x} \pi_{k_u, k_x, \theta}(x, u) \langle f_{k_u, \theta}(u), g_{k_x, \theta}(x) \rangle \quad (6)$$

These logits in Equation 6 can be computed with one *bmm* for inference, and one *mm* for training as will be shown in Section 3.2.

3.1.1 Adaptive Logits Selection. The k_u user-side and k_x item-side embeddings in Equation 6 can be interpreted as (transformations) of user-side and item-side features (e.g., [1, 9]). The drawback is that there could be hundreds to thousands of such features in production settings, which makes computing $k_u \times k_x$ logits prohibitively expensive. We control computational cost as follows:

- *user-side / context-side.* We reduce computational cost by compressing the feature embeddings *adaptively* (‘Adaptive Embedding Compression’ in Figure 1(b)). We transform the k'_u embeddings v'_i ’s into k_u embeddings v_i ’s of the same dimensionality:

$$v_i = \sum_{0 \leq j < k'_u} w_{j,i} v'_j. \quad (7)$$

- *item-side.* Besides computational challenge, accelerator memory limits how many embeddings we can store. On a 16GB accelerator and with a 10M corpora where $d = 128$, we can only cache 6 item-side embeddings with FP16. Besides using Equation 7 for compression, we can generate some item-side embeddings on the fly with learned MLPs, which trades off compute for memory.

3.2 Shared Negatives and Gating Function Decomposition

So far we have omitted discussions about how $\pi_{k,\theta}(x, u)$ is defined. Modern neural retrieval models are commonly trained with sampled softmax losses [3, 26, 36, 45], and utilizing more negatives generally improves model quality [45]. Even if we share the negatives for samples in a mini-batch, the large number of negatives can still make popular gating/attention architectures which are originally designed for ranking models prohibitively expensive in terms of computation and memory. For example, in AttentionFM [43], the attention weights for logits are computed as $\alpha_{ux} = \text{MLP}(v_u \odot v_x)$ where $\odot$ is element-wise dot product. With the extra item dimension in large batch negative training, the computational cost for attention weights becomes $O(BX(k_u k_x)^2 D_f)$ for one linear layer, where B is mini batch size, X is number of negatives, and D_f is the feature embedding dimension. This requires 35184.4 GFLOPs and 274.9GB HBM for configurations discussed in Table 2 ($D_f = 256$).

Therefore, in addition to sharing negatives across users in a mini-batch as done in standard negative sampling [3, 40, 45] and removing element-wise dot products, we further propose to rewrite the gating function with decomposition to reduce the computational cost of materializing and performing matrix multiplications over large (B, X, D) tensors. This is done by parameterizing $\pi_\theta(x, u)$ as

$$\pi_\theta(x, u) = \sigma_\theta \left(\pi_\theta^X(x), \pi_\theta^U(u), \pi_\theta^{XU}(x|u) \right) \quad (8)$$

where $\pi_\theta^X(x)$ is a DNN that takes item-side feature of dimensionality D^X as input, $\pi_\theta^U(u)$ is a DNN that takes user-side feature of dimensionality D^U as input, and $\pi_\theta^{XU}(x|u)$ takes cross features of dimensionality D^{XU} (e.g. $k_x \cdot k_u$) as input, and $\sigma_\theta(\cdot, \cdot, \cdot)$ is some nonlinearity (e.g., softmax after linear combination of inputs). Assuming we use two-layer MLPs with K (or K^U, K^X, K^{XU}) as hidden dim and L as output dim, the original computational cost of $\pi_\theta(x, u)$ is $O(BXK(D+L))$, which is reduced to $O(BK(D^U+L)+XK(D^X+L))$ with gating function decomposition. With de-

Table 2: Effect of $\pi_\theta(x, u)$ decomposition. $B = 2048, X = 4096, D = 1024 (D^U = 768, D^X = 128, D^{XU} = 128), K = 256, L = 128$.

	GFLOPs	HBM (fp32)
w/o gating decomposition	2473.9	44GB
w/ gating decomposition	1101.0 (-55.5%)	16GB (-63.6%)

composition of user, item, user-item tensors, we are able to reduce the computational cost of the most expensive part (with $D^{XU} \ll D$) while maintaining model quality. We show the benefit of this approach for practical settings in Table 2.

Finally, we regularize $\pi_\theta(x|u)$ by forcing it to take the form of a probability distribution per (user, item) pair, i.e., $\sum_k \pi_{k,\theta}(x|u) = 1$. We found that applying dropout on top of this distribution to be helpful, which encourages more balanced mixture component utilization similar to Mixture-of-Experts [14, 16].

3.3 Component-level Hypersphere Embeddings

The large number of component-level embeddings used in Equation 4 and 6 could introduce two problems, preventing generalization of the model to the torso and long tail in retrieval setting:

- The gating function $\pi_\theta(x, u)$ can make component-level embeddings undertrained (esp. in online training settings), which in turn introduces training instability over time.
- They may exacerbate overfitting or require significantly more negatives (more compute) to properly fit the model.

We propose L2 normalizing the component embeddings as a simple and effective regularization to fix the problem. The intuition is as follows. L2 normalization forces all MoL component embeddings in Equation 6 to lay on the surface of a d -dimensional hypersphere. Denote the volume of a d -dimensional hypersphere by $V(d)$. We have $V(d) \geq \frac{1}{\sqrt{d-1}} V(d-1)$; as d grows to infinity, the majority of the volume of a unit hypersphere lies near its ‘equator’ which is in turn a $(d-1)$ dimensional hypersphere. Further, the majority of a unit hypersphere’s volume lies near its surface. This property encourages component-level hypersphere embeddings, even if undertrained, to be near-orthogonal to each other: $\lim_{d \rightarrow \infty} \langle f_{k_u}(u), g_{k_x}(x) \rangle / \|f_{k_u}(u)\|_2 \|g_{k_x}(x)\|_2 = 0$. In contrast, without normalization, assume the dimensions of each embedding are i.i.d. in $[-1, 1]$, the limit of the above equation goes to infinity. Formally, the modified MoL setup can be found in Equation 9, where the term τ ($\tau > 1$) rescales post softmax output range back to $(0, 1)$, similar to temperature in L2-normalized softmax.

$$\phi_{\text{MoL}_{\text{final}}}(x, u) = \sum_{k_u, k_x} \pi_{k_u, k_x, \theta}(x, u) \left\langle \frac{f_{k_u, \theta}(u)}{\|f_{k_u, \theta}(u)\|_2}, \frac{g_{k_x, \theta}(x)}{\|g_{k_x, \theta}(x)\|_2} \right\rangle / \tau \quad (9)$$

Besides solving the problem of undertrained component-level embeddings, the other nice property of Equation 9 is that we force MoL logit components to measure angular distances instead of both vector magnitudes and angular distances, which helps avoiding degrading the model to only learn about popular items given impressions on items generally follow a power-law distribution. We present ablation studies in Section 5.1.3.

3.4 Final MoL Algorithm

Pseudocode for the final mixture-of-logits model using dot products as mixture components can be found in Algorithm 1, which is also illustrated in Figure 1(b). In practice, we use simple 2-layer MLPs for *queryWeightFn*, *itemWeightFn*, and *crossWeightFn* and set their output size to be $k_u \cdot k_x$. We use a simple *combinationFn*, $f(uw, xw, cw) = uw \cdot xw + cw$ followed by SiLU nonlinearity [7].

Algorithm 1 Mixture of Logits (MoL) Algorithm.

```

1: procedure MoL(data, query, k, k', ku, kx, d,  $\tau$ )
2:   userEmbs  $\leftarrow$  L2NORM(userEmbProj(query))
3:   itemEmbs  $\leftarrow$  [] ▷ Of shape (kx, d)  $\times$  k'
4:   for x  $\in$  data do ▷ Cachable
5:     itemEmbs.append(L2NORM(itemEmbProj(x)))
6:   end for
7:   cl  $\leftarrow$  mm(userEmbs, itemEmbs.reshape(−1, d).t())/ $\tau$ 
8:   cl  $\leftarrow$  cl.view(ku, k', kx).permute(1, 0, 2).reshape(k', −1)
9:   gatingWeights  $\leftarrow$  DECOMPOSEDGATING(data, query, cl)
10:  similarities  $\leftarrow$  (gatingWeights · cl).sum(−1)
11:  return argsort(similarities)[ $:$  k]
12: end procedure
13: procedure DECOMPOSEDGATING(data, u, allLogits)
14:  uWeights  $\leftarrow$  userWeightFn(u)
15:  xWeights  $\leftarrow$  itemWeightFn(data) ▷ Cachable
16:  crossWeights  $\leftarrow$  crossWeightFn(allLogits)
17:  return softmax(combineFn(
18:    uWeights, xWeights, crossWeights)) ▷ Of shape (k', ku · kx)
19: end procedure

```

The time complexity for caching item-side computations in MoL is $|\mathbb{X}| \cdot (O(\text{itemEmbProj}) + O(\text{itemWeightFn}))$, which is negligible on accelerators. Assuming all 2-layer MLPs have a hidden dimension of d_h , user embedding input dimension is d_u , and all cachable computation are cached, the MoL stage has a computational cost of $O(\text{userEmbProj}) + O(k_u \cdot d + k' \cdot k_u \cdot k_x \cdot d) + O(\text{userWeightFn}) + k' \cdot O(\text{crossWeightFn}) + k' \cdot O(\text{combineFn}) + O(k' \cdot k_u \cdot k_x)$. This cost is dominated by the user-item cross parts: $O(k' \cdot k_u \cdot k_x \cdot d + k' \cdot O(\text{crossWeightFn}) + k' \cdot O(\text{combineFn})) = O(k' k_u k_x (d + d_h))$.

4 HIERARCHICAL RETRIEVAL AND INFRASTRUCTURE OPTIMIZATIONS

We made the following optimizations to enable large-scale MoL based retrieval model training and serving in production.

4.1 H-indexer Design and Optimization

In the hierarchical retrieval design (Section 2.2), the first stage finds a large number of candidates ($k' \sim 10^5$) with the goal of not missing any candidates that would have originally been selected by the final similarity function in top $k = 100 \sim 1000$. The high value of k' makes blockwise based top-k algorithms such as FAISS [15] and TPUKNN [2] less applicable, where small k' s (≤ 2048) are preferred due to register size limit or algorithm design. We therefore propose an approximate top-k design for very large k s, *h-indexer*, with pseudocode in Algorithm 2. We discuss optimizations used below.

Algorithm 2 h-indexer top-k algorithm for large k s (k' s). λ is the selection ratio for estimating top k similarity threshold.

```

1: procedure H-INDEXER(data, query, k,  $\lambda$ )
2:   randIndices  $\leftarrow$  PERMUTATION(0.. $|data| - 1$ )[ $:$   $\lambda$ ]
3:   sampledSimilarities  $\leftarrow$  []
4:   for x  $\in$  data[randIndices] do
5:     sampledSimilarities.append(DOTPRODUCT(x, query))
6:   end for
7:   t  $\leftarrow$  NTHELEMENT(sampledSimilarities,  $k/|data| \cdot \lambda$ )
8:   indices  $\leftarrow$  []
9:   for (indice, x)  $\in$  ENUMERATE(data) do
10:    if DOTPRODUCT(x, query)  $>$  t then
11:      indices.append(indice)
12:    end if
13:  end for
14:  return indices
15: end procedure

```

4.1.1 dot-product optimization. To control the computational cost of dot-product stage, we used a low embedding dimension (64). Meanwhile, dot product is memory-bounded and its arithmetic intensity — a measure indicating the degree of an op being compute-bound over memory-bound [42] — only scales with batch size and reversely with input byte-width (byteW),

$$A.I. = \frac{B \times X \times D' \times 2FLOPs}{(B \times D' + X \times D' + B \times X) \times \text{byteW}} \approx \frac{2B}{\text{byteW}} \quad (10)$$

where corpus size (X) and embedding dimension (D) are far greater than batch size (B). With this observation, we boosted the GPU utilization by batching user requests and applying INT8 quantization. Dot product shows 1.5 $\times$ speed boost compared to the half precision thanks to INT8. Further, the output of INT8 GEMM is INT32, which can be digested by top-k selection directly without applying scale/bias to de-quantize to FP32 as in the regular practice.

4.1.2 approximate top-k. Finding top- k' in a corpus with X items has a time complexity of $\Omega(X \log k')$, which is exacerbated by the large k' number. We therefore adopted an approximate top-k approach to randomly sample a small portion of the corpus to estimate the threshold for top k' items, and subsequently finding up to k' samples that meet that threshold. This reduces time complexity to $\Omega(X + rX \log k')$ with the sampling ratio r ranges from 0.01 to 0.1. In production settings, this change is $\sim 2.5\times$ faster than exact top-k and reduces h-indexer latency by 30%.

4.1.3 index select. We further optimized the index select op after top-k selection in Algorithm 2. Fetching and concatenating 10^5 item embeddings out of a 10M pool is expensive with a throughput of around 0.15 TB/s. We designed an optimized GPU kernel with $2\times$ throughput by leveraging data locality and efficient caching [20].

4.2 Op Fusion for Latency and Memory Saving

We also performed Op fusion to further reduce the serving cost for the MoL stage and achieved 30% performance improvement. While GEMM ops are highly optimized on GPUs, Armdahl's law dictates that non-GEMM ops would become a bottleneck for the end-to-end latency. The read/write cost of input/output tensors

between HBM and GPU’s caches is significantly exacerbated by the large item size (10^5). Fusing consecutive non-GEMM ops, such as $\text{SiLU}(x) = x \cdot \text{Sigmoid}(x)$, and fusing non-GEMM ops as epilogues of GEMM could efficiently reduce R/W induced latency. Op fusion can also reduce memory usage for MoL. For example, the op below requires expansion of input tensors by $B = 8$ or $K' = 10^5$ for the operands in SoftMax, which can be avoided in a fused kernel.

$$\text{Out} = [B, K', D] \times \text{SoftMax}([B, D] + [K', D])$$

We also fused line 7 and 8 in Algorithm 1. Since the size of cl is much larger than userEmbs and itemEmbs , fusing the GEMM op and the permute op can greatly save memory and improve throughput.

4.3 BFLOAT16 Computation

Although FP16 training has demonstrated $2\times$ throughput than FP32 training [27], the range of FP16 (1-5-10) is insufficient for the backwards gradients in training. The compensating loss scaling technique discards the overflowed iteration unpredictably and results in poor reproducibility for production. We therefore explored the training in BF16 with the same range as FP32. We further aggressively applied BF16 precision to ops like SoftMax, LayerNorm, and Division and observed no apparent degradation of the final metrics. It is important to perform these non-GEMM ops in BF16 to avoid the time- and memory-costly conversions between BF16 and FP32. Once the model is trained in BF16, we inference it in the FP16 precision. The $\text{BF16} \rightarrow \text{FP16}$ conversion introduces no quantization noise as FP16 has more mantissa bits than BF16. Eventually, training throughput is increased by 22% with BF16 computations.

4.4 FP8 Quantized Communication

Another innovation we performed in this study is aggressive quantization of All2All traffic to the FP8 format [28, 35]. As a costly bottleneck for recommendation model training, All2All communication that switches between data parallelism and model parallelism is especially expensive for our retrieval model thanks to the complex user embeddings from features and from sequential encoding. FP8 quantization has only 8-bit bitwidth and can handle both forward activations and backward gradients with dynamic scaling. We created a customized FP8 rowwise quantization kernel [20] to quantize all2all tensors before the communication. Since All2All does not require any arithmetic operations, the FP8 rowwise quantization kernel can be executed by A100 GPU that does not even support FP8 arithmetics. The resulting final metrics show no degradation and gained 16% throughput over 16-bit All2All communication. In combination, quantized training with BF16 computation with FP8 communication improved throughput by 37%.

5 EXPERIMENTS

We analyze the proposed techniques on both public datasets and large scale industrial datasets. Our proposed techniques are agnostic to the exact retrieval model setup, and apply as long as we need to model (user, item) similarities. We hence consider two settings:

- *Non-sequential methods.* This is a standard retrieval setup where the goal is to model the probability of recommending a candidate item given a user as in Equation 1, and is commonly used in various retrieval and recommendations settings [3, 8, 11, 25, 26, 48, 50, 51]. Note that while the model setup is not fully sequential,

Table 3: Data statistics (after preprocessing and truncation)

	# of users	# of items	avg. ratings per item	avg. ratings per user
ML-1M	6,040	3,649	181.59	109.70
ML-20M	138,493	24,186	528.40	92.28
Beauty	40,226	54,243	6.40	8.62
Games	29,341	23,310	11.97	9.15
Books	694,897	674,044	11.51	11.61

sequential encoders are commonly used in these setups to provide features as user-/context-side encoders [25, 26, 48, 50, 51].

- *Sequential methods.* These methods predict next item based on sequential user-item interactions [18, 34, 44]. User interactions are modeled as a (higher-order) Markov chain.

We tuned hparams for all non-dot-product setups (MLP, NeuMF [11], DeepFM [9], MoL) so that they use similar compute. We omitted AttentionFM [43] since it cannot fit into 80GB HBM for most configurations due to AFM’s usage of element-wise products combined with large number of negatives, as discussed in Section 3.2.

5.1 Public datasets

We consider MovieLens (1M and 20M variants), Amazon Reviews (Beauty, Games, and Books subsets) for consistency with prior work [18, 26, 34, 48, 50]. We follow standard preprocessing, including discarding users and items with fewer than 5 related actions, and truncating user behavioral sequences to 200 (MovieLens) / 50 (Amazon) [18, 34]. Statistics for all datasets can be found in Table 3.

For these datasets, due to the popularity of sequential methods in recent years, we apply our proposed approach on top of a sequential baseline, SASRec [18]. SASRec trains the model to predict the next item in an autoregressive manner. SASRec represents one of the best known approaches on public datasets; for instance, on ML-1M, our SASRec implementation achieves .223 HR@10 (Table 4), on par with best result (.224) from [4], and on Amazon Games, our impl’s HR@10 is .0794, which is within 10% of the best result in [4].

5.1.1 Evaluation Methodologies. Prior work have reported metrics based on different sampling methodologies; e.g., Kang et al. used 100 randomly sampled negatives [18], and Sun et al. used 100 popularity-weighted sampled negatives [34]. For consistency with standard retrieval settings, we report metrics evaluated over the entire corpus, which is also done in [4]. We omit *h-indexer* given the corpus are small. We focus on HR@10 for MovieLens and HR@200 for Amazon Reviews based on prior work.

5.1.2 Experiments. We consider the baseline setting (binary cross entropy loss on top of dot products, BCE) and also other settings using sampled softmax (SS), including Dot Product, MLP, NeuMF, DeepFM, and MoL. We train all variants with Adam [21], a learning rate of 0.001, and a mini batch size of 128. For sampled softmax loss, we sample 128 negatives for denser datasets like ML-1M, ML-20M, and Amazon Books, and 512 negatives otherwise. Given that there exists only one group of features in the sequential setting, we use learned MLPs to project user embedding at step t to k_u embeddings, and item embedding to k_x embeddings for MoL and DeepFM. For other parameters, we follow reported optimal settings

when those exist, and grid-searched parameters otherwise. Detailed configurations are in Appendix A. All models are implemented in PyTorch and are run on a single A100 80GB GPU.

Experiment results can be found in Table 4 and Table 6. *Dot product + SS* significantly outperforms *baseline (BCE)* in all settings, confirming the importance of sampled softmax loss in retrieval setting. Non-dot product based methods (e.g., NeuMF and MoL) tend to significantly outperform dot products on datasets that are not extremely sparse (ML-1M, ML-20M, and Amazon Books). Dot products remain competitive for highly sparse datasets, e.g., Beauty and Games (avg < 10 interactions per item).

We also observe that we can indeed obtain high rank $\phi(u, x)$ distributions with non-dot-product methods, as shown in Table 5.

5.1.3 Ablation Studies. We run the following ablation studies to understand how different components in MoL contribute to overall performance. We fix the baseline as the default MoL configuration, which differs from Table 4 and 6 in that we enabled component-level hypersphere embeddings for Amazon Beauty and Games datasets.

- *no-l2-norm*: Ablate component-level hypersphere embeddings.
- *no-gating-dropout*: Ablate dropout in gating.
- *50%- $k_u \times k_x$* : Use 50% fewer mixture components for k_u and k_x .
- *25%-negatives*: Use 75% fewer negative examples.

Results can be found in Table 7. Most components show consistent gains except on the sparse Amazon datasets. Those that consistently contribute to gains (> 2% are underlined) include component-level hypersphere embeddings, gating dropout, and number of negatives.

5.2 Industrial Datasets

We next consider a non-sequential model setup, and evaluate model performance on a proprietary industrial dataset against strong baselines. All models share the same features, labels, and losses (incl. number of negatives), and all non-MoL setups use similar compute as the most expensive MoL setup (32×4). Equation 7 was used to compress user-side and item-side embeddings for DeepFM [9].

Results are shown in Table 8. With sufficient model capacity, our proposed MoL model outperforms all baselines under the same compute budget. The best MoL model improves HR@50 by 23.3% compared with Dot product and 5.0% compared with DeepFM.

5.2.1 Hierarchical Retrieval. We next evaluate how our hierarchical retrieval design affects model quality and scalability. Figure 3 compares the performance of MoL with and without h-indexer. We observe that h-indexer does not degrade recall for suitably chosen values of K' (e.g., 10^5). While MoL + h-indexer with $K' = 10^5$ has almost the same recall as the MoL-only model ($K' = X$), its throughput is less sensitive to corpus size X and enables us to scale up to 100M items on a single GPU.

5.2.2 Effect of Different Mixture Components. We analyze the role of mixture components to understand how model performance scales with model capacity in Table 8. As we increase number of mixture components from 8×4 to 32×4, we can observe significant performance improvement in each scale-up. Meanwhile, the performance of Dot product starts to saturate at dim=256.

5.2.3 Popularity Bias. We observe that MoL and other non-dot product methods also reduce popularity bias. We plot a histogram

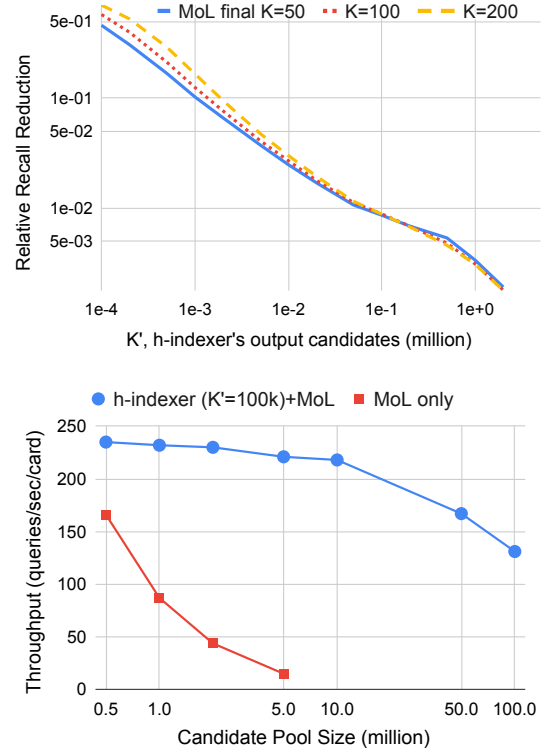


Figure 3: Recall and serving cost of the two-stage h-indexer/MoL model: (a) relative recall ratio vs MoL-only model under varying k' over 10M items, and (b) throughput of two-stage vs one-stage over different pool sizes.

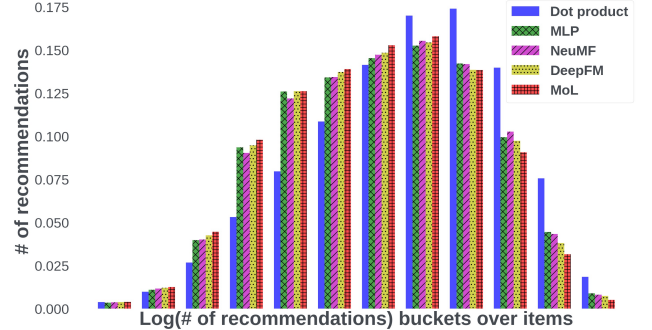


Figure 4: Distributions of recommendations over log-scaled recommendation frequency buckets. Lower bars in higher buckets indicates that fewer head items are shown.

to show the distributions of recommended items in Figure 4. Items recommended by MLP, NeuMF, DeepFM, and MoL have more coverage in tail segments and are less biased towards popular ones, with MoL showing the most significant reduction in head buckets.

5.3 Online Experiments

We conducted A/B testing for the proposed setup on a major recommendation surface at Meta serving billions of users. Over a period of 7 days, we observed a significant gain of +3.2% in our main

Table 4: MovieLens (-1M and -20M) results on top of sequential methods.

	method	hr@1	hr@10	hr@50	hr@500	mrr
ML-1M	baseline (BCE)	0.0409	0.2230	0.4929	0.8594	0.1018
	Dot product + SS	0.0669 (+63.6%)	0.2821 (+26.5%)	0.5435 (+10.3%)	0.8618 (+0.3%)	0.1363 (+33.9%)
	MLP + SS	0.0576 (+40.9%)	0.2805 (+25.8%)	0.5419 (+9.9%)	0.8551 (-0.5%)	0.1290 (+26.8%)
	NeuMF + SS	0.0545 (+33.2%)	0.2854 (+28.0%)	0.5465 (+10.9%)	0.8672 (+0.9%)	0.1264 (+24.2%)
	DeepFM (8×8) + SS	0.0626 (+53.0%)	0.2856 (+28.1%)	0.5526 (+12.1%)	0.8644 (+0.6%)	0.1335 (+31.2%)
	MoL (8×8) + SS	0.0675 (+65.2%)	0.3079 (+38.1%)	0.5657 (+14.8%)	0.8677 (+1.0%)	0.1433 (+40.8%)
ML-20M	baseline (BCE)	0.0275	0.1756	0.4467	0.8543	0.0782
	Dot product + SS	0.0620 (+125.1%)	0.2862 (+63.0%)	0.5488 (+22.9%)	0.8721 (+2.1%)	0.1344 (+71.9%)
	MLP + SS	0.0687 (+149.4%)	0.2936 (+67.2%)	0.5556 (+24.4%)	0.8756 (+2.5%)	0.1416 (+81.1%)
	NeuMF + SS	0.0645 (+134.1%)	0.2911 (+65.7%)	0.5548 (+24.2%)	0.8760 (+2.5%)	0.1376 (+76.0%)
	DeepFM (8×8) + SS	0.0657 (+138.7%)	0.2932 (+67.0%)	0.5542 (+24.1%)	0.8763 (+2.6%)	0.1394 (+78.2%)
	MoL (8×8) + SS	0.0758 (+175.3%)	0.3114 (+77.3%)	0.5727 (+28.2%)	0.8847 (+3.6%)	0.1528 (+95.3%)

topline metric and +0.22% completion rate compared with baseline model.

We also compared the latency and throughput between the older production CPU model and the new MoL + h-indexer model on GPUs. The median serving GPU latency of our proposed architecture is 30ms, comparable with optimized CPU MIPS baseline (20ms). The latency regression is a result of request batching optimization, where we let the service wait for up to 10 ms to construct a batch from live requests. Despite high FLOPs, our design does not significantly increase serving cost due to its high arithmetic intensity. A comparable GPU MIPS implementation has 13ms latency (before batching) and 1.8× throughput vs MoL + h-indexer.

6 RELATED WORK

Dot Product as an Embedding Combiner. Dot products have been widely used to combine model outputs with class information in seminal work across image classifications (e.g., AlexNet [23]), natural language processing (e.g., Transformers [37] and BERT [5]), and information retrieval (e.g., YouTube DNNs [3]).

Recently, softmax layer is observed to be a bottleneck for some NLP applications [46], and averaging multiple softmaxes has been found to be helpful in improving model perplexity. Averaging softmaxes, however, requires computation of normalization factor for each softmax, which is costly for NLP vocabularies in the range of $10^4 - 10^5$ [46] and cost-prohibitive for modern IR settings that need to deal with millions to billions of items [6, 39]. Our algorithm not only reduces computational cost, but also enables better efficiency via embedding sharing as discussed in Section 3.1.

Second- and Higher-Order Interaction Modeling. Motivated by observations such as people often downloading food delivery apps at mealtime [9], feature interactions have been recognized as an important factor for achieving good recommendation performance

in various works, including Factorization Machines (FMs) [31] and their deep neural network variants [9, 43], product-based NNs [29], DCNs [41], and so on. The applications of these approaches have been limited to CTR predictions, or ranking. We believe that this is largely due to the difficulty of generalizing MLPs to long tail as discussed by Rendle et al. in [32]. Compared with Attention FMs [43] and PNNs [29], MoL leverages adaptive embedding compression and gating decomposition to significantly reduce compute and memory usage, and l2 norm'd features to improve generalization.

Multi-Interest Recommendations and Short-/Long-Term Interests. A line of work related to interaction modeling is multi-interest embeddings [25, 26, 36] which aims to capture diverse preferences of a user over different topics and time periods. For cases where the embedding combiner is not user- and item- dependent, like SINE and SDM [26, 36], the resulting model could be more expressive but still necessarily low rank as we've shown in Equation 5. Item-dependent approaches like MIND [25] can be viewed as a special form of MoL where there is exactly one item-side embedding and gating function takes a specific form (e.g., dot-product attention).

Learned Discrete Structures for Retrieval. Yet another approach is to learn explicit structures so that retrieval can be viewed as a root-to-leaf traversal, beam search, or operating on top of binary codes directly [8, 19, 50–52]. In particular, TDM [51] first enabled expressive neural networks to be applied at retrieval time. These approaches are more sensitive to hyperparameters, and if heavy branching is involved, can be hard to scale on accelerators.

Accelerating Inference for Retrieval. In the traditional dot product retrieval setting, or MIPS (Maximum Inner Product Search), product quantization and hashing techniques [10, 33] have been well studied and are widely used in the non-accelerator retrieval settings.

Partitioning the item space is a complementary approach to speed up inference by reducing search space [8, 22, 24, 30, 47, 50, 51]. Search space can be partitioned with clustering [24, 47], spatial-partitioning trees [22, 30], or with approaches where the partition strategy is learned [8, 50, 51]. This line of work can be viewed as alternatives to *h-indexer* for hierarchical retrieval settings.

A recent line of work investigates efficient MIPS-based KNN retrieval on accelerators [2, 15]. There have not been significant work on non-MIPS setups on accelerators to date.

Table 5: Rank of learned $\phi(u, x)$ on ML-1M ($d = 50$). Note that a high rank matrix does not always imply high hit rate.

	dot product	MLP	NeuMF	DeepFM	MoL
Rank	50	227	386	304	1864

Table 6: Amazon Reviews (Beauty, Games, and Books) results on top of sequential methods.

	method	hr@10	hr@50	hr@200	hr@500	mrr
Beauty	baseline (BCE)	0.0256	0.0743	<u>0.1604</u>	0.2370	0.0126
	Dot product + SS	0.0655 (+155.4%)	0.1379 (+85.5%)	0.2271 (+41.6%)	0.2914 (+23.0%)	0.0333 (+164.4%)
	MLP + SS	0.0393 (+53.4%)	0.1005 (+35.2%)	<u>0.1864 (+16.2%)</u>	0.2465 (+4.0%)	0.0189 (+49.8%)
	NeuMF + SS	0.0535 (+108.9%)	0.1247 (+67.8%)	<u>0.2152 (+34.2%)</u>	0.2818 (+18.9%)	0.0264 (+109.6%)
	DeepFM (4×4) + SS	0.0494 (+92.9%)	0.1271 (+71.0%)	<u>0.2238 (+39.5%)</u>	0.2953 (+24.6%)	0.0241 (+91.3%)
	MoL (4×4) + SS	0.0541 (+111.0%)	0.1288 (+73.3%)	<u>0.2246 (+40.0%)</u>	0.2961 (+25.0%)	0.0262 (+107.5%)
Games	baseline (BCE)	0.0794	0.2209	<u>0.4217</u>	0.5642	0.0383
	Dot product + SS	0.1246 (+57.0%)	0.2771 (+25.4%)	<u>0.4505 (+6.8%)</u>	0.5736 (+1.7%)	0.0600 (+56.7%)
	MLP + SS	0.1066 (+34.3%)	0.2537 (+14.9%)	<u>0.4293 (+1.8%)</u>	0.5582 (-1.1%)	0.0498 (+30.1%)
	NeuMF + SS	0.1175 (+48.0%)	0.2722 (+23.2%)	<u>0.4479 (+6.2%)</u>	0.5755 (+2.0%)	0.0559 (+46.0%)
	DeepFM (4×4) + SS	0.1216 (+53.2%)	0.2817 (+27.5%)	<u>0.4690 (+11.2%)</u>	0.5965 (+5.7%)	0.0576 (+50.5%)
	MoL (4×4) + SS	0.1221 (+53.9%)	0.2834 (+28.3%)	0.4713 (+11.8%)	0.5964 (+5.7%)	0.0586 (+53.2%)
Books	baseline (BCE)	0.0247	0.0660	<u>0.1370</u>	0.2079	0.0123
	Dot product + SS	0.0317 (+28.3%)	0.0814 (+23.3%)	<u>0.1588 (+15.9%)</u>	0.2295 (+10.4%)	0.0156 (+26.8%)
	MLP + SS	0.0297 (+20.2%)	0.0759 (+15.0%)	<u>0.1500 (+9.5%)</u>	0.2190 (+5.3%)	0.0144 (+17.1%)
	NeuMF + SS	0.0358 (+45.0%)	0.0871 (+32.0%)	<u>0.1644 (+20.0%)</u>	0.2338 (+12.5%)	0.0178 (+44.7%)
	DeepFM (8×8) + SS	0.0361 (+46.2%)	0.0905 (+37.1%)	<u>0.1706 (+24.5%)</u>	0.2414 (+16.1%)	0.0179 (+45.5%)
	MoL (8×8) + SS	0.0388 (+57.1%)	0.0934 (+41.5%)	0.1751 (+27.8%)	0.2479 (+19.2%)	0.0194 (+57.7%)

Table 7: Ablation studies on top of MoL and sequential methods.

	ML-1M (HR@10)	ML-20M (HR@10)	Beauty (HR@200)	Games (HR@200)	Books (HR@200)
MoL	0.3079	0.3114	0.2088	0.4618	0.1751
no-l2-norm	<u>0.2894 (-6.0%)</u>	<u>0.3032 (-2.6%)</u>	<u>0.2246 (+7.5%)</u>	<u>0.4713 (+2.0%)</u>	<u>0.1695 (-3.2%)</u>
no-gating-dropout	<u>0.2964 (-3.8%)</u>	<u>0.3100 (-0.5%)</u>	<u>0.2148 (+2.8%)</u>	<u>0.4592 (-0.6%)</u>	<u>0.1674 (-4.4%)</u>
50%- $k_u \times k_x$	<u>0.3041 (-1.2%)</u>	<u>0.3022 (-2.9%)</u>	<u>0.2076 (-0.6%)</u>	<u>0.2076 (-0.6%)</u>	<u>0.1719 (-1.8%)</u>
25%-negatives	<u>0.2972 (-3.5%)</u>	<u>0.2907 (-6.7%)</u>	<u>0.2046 (-2.0%)</u>	<u>0.4627 (+0.2%)</u>	<u>0.1808 (+3.3%)</u>

7 CONCLUSIONS AND FUTURE WORK

We have presented new algorithms, including *mixture of logits* and *h-indexer* that enable non-dot-product retrieval models to run efficiently on accelerators with latency and throughput comparable to MIPS retrieval setups. These algorithms, which are compatible with

various retrieval setups, significantly outperform baselines on both public and large industrial datasets with better recommendation quality and reduced popularity bias.

Our work lays the foundation for many interesting future work for non-dot-product based neural retrieval models. For instance, one direction is to make the mixture components in MoL arbitrarily complex neural networks instead of dot products. Another direction could be to make a cascading retrieval setup end-to-end learnable.

Table 8: Offline results on a large-scale industrial dataset.

method	HR@10	HR@50	HR@100	HR@200
Dot product (d=64)	0.0843	0.2563	0.3712	0.5026
Dot product (d=128)	0.0882	0.2676	0.3856	0.5204
Dot product (d=256)	0.0916	0.2761	0.3954	0.5305
Dot product (d=512)	0.0925	0.2754	0.3949	0.5299
MLP	0.0938	0.2656	0.3770	0.5054
NeuMF	0.1087	0.2976	0.4148	0.5479
DeepFM	0.1199	0.3242	0.4465	0.5810
MoL (16×2)	0.1212	0.3221	0.4436	0.5768
MoL (8×4)	0.1221	0.3232	0.4440	0.5763
MoL (16×4)	0.1250	0.3283	0.4502	0.5834
MoL (32×4)	0.1304	0.3404	0.4628	0.5961

8 ACKNOWLEDGEMENTS

We'd like to thank Yinghai Lu, Andrew Tulloch, Pawel Garbacki, Jongsoo Park, Jeff Johnson, Dmytro Ivenchko, Yi Wang, Rui Zhang, Xianjie Chen for various discussions, Jie Hua, Michael He, Pengchao Wang, Chao Yang, Jiacheng Feng, Rui Jian, Hao Lin, Brandon Burroughs, Shripad Thite, Eric Xu, Qing Liu, Jeevan Gyawali, Lei Chen, Bi Xue, Mengchi Zhang, Jianyu Huang, Haixin Liu, Jiecao Yu, Sarunya Pumma, Dhruv Choudhary, Cheng Cheng, Zhiyun Zhang, Will Feng, Fanny Yang, Hao Lu, Wenlei Xie, Bin Wen, Emily Sun, Tianshu Bao, Zexi Mao, Bugra Akyildiz, Hui Zhang, Xinyao Hu, Yun Mao, Kaushik Veeraraghavan for infra/product support, and Shilin Ding, Lin Qiao, Hong Yan, Andre Rohe, Lars Backstrom for overall support of the project.

REFERENCES

- [1] Heng-Tze Cheng, Levent Koc, Jeremiah Harmsen, Tal Shaked, Tushar Chandra, Hrishii Aradhya, Glen Anderson, Greg Corrado, Wei Chai, Mustafa Ispir, Rohan Anil, Zakaria Haque, Lichan Hong, Vihan Jain, Xiaobing Liu, and Hemal Shah. 2016. Wide & Deep Learning for Recommender Systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems (DLRS 2016)*. 7–10.
- [2] Felix Chern, Blake Hechtman, Andy Davis, Ruiqi Guo, David Majnemer, and Sanjiv Kumar. 2022. TPU-KNN: K Nearest Neighbor Search at Peak FLOP/s. In *Advances in Neural Information Processing Systems*.
- [3] Paul Covington, Jay Adams, and Emre Sargin. 2016. Deep Neural Networks for YouTube Recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems (RecSys '16)*. 191–198.
- [4] Alexander Dallmann, Daniel Zoller, and Andreas Hotho. 2021. A Case Study on Sampling Strategies for Evaluating Neural Sequential Item Recommendation Models. In *Proceedings of the 15th ACM Conference on Recommender Systems (RecSys '21)*. 505–514.
- [5] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*. 4171–4186.
- [6] Chantat Eksombatchai, Pranav Jindal, Jerry Zitao Liu, Yuchen Liu, Rahul Sharma, Charles Sugnet, Mark Ulrich, and Jure Leskovec. 2018. Pixie: A System for Recommending 3+ Billion Items to 200+ Million Users in Real-Time. In *Proceedings of the 2018 World Wide Web Conference (WWW '18)*. 1775–1784.
- [7] Stefan Elfving, Eiji Uchibe, and Kenji Doya. 2017. Sigmoid-Weighted Linear Units for Neural Network Function Approximation in Reinforcement Learning. *Neural networks: the official journal of the International Neural Network Society* 107 (2017), 3–11.
- [8] Weihao Gao, Xiangjun Fan, Chong Wang, Jiankai Sun, Kai Jia, Wenzhi Xiao, Ruofan Ding, Xingyan Bin, Hui Yang, and Xiaobing Liu. 2021. Learning An End-to-End Structure for Retrieval in Large-Scale Recommendations. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM '21)*. 524–533.
- [9] Huifeng Guo, Ruiming Tang, Yunming Ye, Zhenguo Li, and Xiuqiang He. 2017. DeepFM: A Factorization-Machine Based Neural Network for CTR Prediction. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI'17)*. 1725–1731.
- [10] Ruiqi Guo, Sanjiv Kumar, Krzysztof Choromanski, and David Simcha. 2016. Quantization based Fast Inner Product Search. In *Proceedings of the 19th International Conference on Artificial Intelligence and Statistics, AISTATS 2016*, Vol. 51. 482–490.
- [11] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu, and Tat-Seng Chua. 2017. Neural Collaborative Filtering. In *Proceedings of the 26th International Conference on World Wide Web (Perth, Australia) (WWW '17)*. 173–182.
- [12] Kurt Hornik, Maxwell Stinchcombe, and Halbert White. 1989. Multilayer feed-forward networks are universal approximators. *Neural Networks* 2, 5 (1989), 359–366.
- [13] Jui-Ting Huang, Ashish Sharma, Shuying Sun, Li Xia, David Zhang, Philip Pronin, Janani Padmanabhan, Giuseppe Ottaviano, and Linjun Yang. 2020. Embedding-Based Retrieval in Facebook Search. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2553–2561.
- [14] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. 1991. Adaptive Mixtures of Local Experts. *Neural Computation* 3, 1 (1991), 79–87.
- [15] J. Johnson, M. Douze, and H. Jegou. 2021. Billion-Scale Similarity Search with GPUs. *IEEE Transactions on Big Data* 7, 03 (Jul 2021), 535–547.
- [16] M.I. Jordan and R.A. Jacobs. 1993. Hierarchical mixtures of experts and the EM algorithm. In *Proceedings of 1993 International Conference on Neural Networks (IJCNN-93-Nagoya, Japan)*, Vol. 2. 1339–1344 vol.2.
- [17] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*. 1–12.
- [18] Wang-Cheng Kang and Julian McAuley. 2018. Self-attentive sequential recommendation. In *2018 International Conference on Data Mining (ICDM)*. 197–206.
- [19] Wang-Cheng Kang and Julian McAuley. 2019. Candidate Generation with Binary Codes for Large-Scale Top-N Recommendation. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (Beijing, China) (CIKM '19)*. 1523–1532.
- [20] Daya Khudia, Jianyu Huang, Protonu Basu, Summer Deng, Haixin Liu, Jongsoo Park, and Mikhail Smelyanskiy. 2021. FBGEMM: Enabling High-Performance Low-Precision Deep Learning Inference. *arXiv preprint arXiv:2101.05615* (2021).
- [21] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations (ICLR'15)*.
- [22] Noam Koenigstein, Parikshit Ram, and Yuval Shavit. 2012. Efficient Retrieval of Recommendations in a Matrix Factorization Framework. In *Proceedings of the 21st ACM International Conference on Information and Knowledge Management (CIKM '12)*. 535–544.
- [23] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. 2012. ImageNet Classification with Deep Convolutional Neural Networks. In *Advances in Neural Information Processing Systems (NIPS'12, Vol. 25)*.
- [24] Chen Li, E. Chang, H. Garcia-Molina, and G. Wiederhold. 2002. Clustering for approximate similarity search in high-dimensional spaces. *IEEE Transactions on Knowledge and Data Engineering* 14, 4 (2002), 792–808.
- [25] Chao Li, Zhiyuan Liu, Mengmeng Wu, Yuchi Xu, Huan Zhao, Pipei Huang, Guoliang Kang, Qiwei Chen, Wei Li, and Dik Lun Lee. 2019. Multi-Interest Network with Dynamic Routing for Recommendation at Tmall. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM '19)*. 2615–2623.
- [26] Fuyu Lv, Taiwei Jin, Changlong Yu, Fei Sun, Quan Lin, Keping Yang, and Wilfred Ng. 2019. SDM: Sequential deep matching model for online large-scale recommender system. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. 2635–2643.
- [27] Paulius Mikičevičius, Sharan Narang, Jonah Alben, Gregory Diamos, Erich Elsen, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. 2018. Mixed Precision Training. In *International Conference on Learning Representations*.
- [28] Paulius Mikičevičius, Dusan Stosic, Neil Burgess, Marius Cornea, Pradeep Dubey, Richard Grisenthwaite, Sangwon Ha, Alexander Heinecke, Patrick Judd, John Kamalu, et al. 2022. FP8 formats for deep learning. *arXiv preprint arXiv:2209.05433* (2022).
- [29] Yanru Qu, Han Cai, Kan Ren, Weinan Zhang, Yong Yu, Ying Wen, and Jun Wang. 2016. Product-Based Neural Networks for User Response Prediction. In *2016 IEEE International Conference on Data Mining (ICDM)*. 1149–1154.
- [30] Parikshit Ram and Alexander G. Gray. 2012. Maximum Inner-Product Search Using Cone Trees. In *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '12)*. 931–939.
- [31] Steffen Rendle. 2010. Factorization Machines. In *2010 IEEE International Conference on Data Mining (ICDM)*. 995–1000. <https://doi.org/10.1109/ICDM.2010.127>
- [32] Steffen Rendle, Walid Krichene, Li Zhang, and John Anderson. 2020. Neural Collaborative Filtering vs. Matrix Factorization Revisited. In *Fourteenth ACM Conference on Recommender Systems (RecSys'20)*. 240–248.
- [33] Anshumali Shrivastava and Ping Li. 2014. Asymmetric LSH (ALSH) for Sublinear Time Maximum Inner Product Search (MIPS). In *Advances in Neural Information Processing Systems*, Vol. 27.
- [34] Fei Sun, Jun Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou, and Peng Jiang. 2019. BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM '19)*. 1441–1450.
- [35] Xiao Sun, Jungwook Choi, Chia-Yu Chen, Naigang Wang, Swagath Venkataramani, Vijayalakshmi Viji Srinivasan, Xiaodong Cui, Wei Zhang, and Kailash Gopalakrishnan. 2019. Hybrid 8-bit floating point (HFP8) training and inference for deep neural networks. *Advances in neural information processing systems* 32 (2019).
- [36] Qiaoyu Tan, Jianwei Zhang, Jiangchao Yao, Ninghao Liu, Jingren Zhou, Hongxia Yang, and Xia Hu. 2021. Sparse-Interest Network for Sequential Recommendation. In *Proceedings of the 14th ACM International Conference on Web Search and Data Mining*. 598–606.
- [37] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17)*. 6000–6010.
- [38] Hao Wang, Xingjian Shi, and Dit-Yan Yeung. 2016. Collaborative Recurrent Autoencoder: Recommend While Learning to Fill in the Blanks. In *Proceedings of the 30th International Conference on Neural Information Processing Systems (NIPS'16)*. 415–423.
- [39] Jizhe Wang, Pipei Huang, Huan Zhao, Zhibo Zhang, Binqiang Zhao, and Dik Lun Lee. 2018. Billion-Scale Commodity Embedding for E-Commerce Recommendation in Alibaba. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '18)*. 839–848.
- [40] Jimpeng Wang, Jieming Zhu, and Xiuqiang He. 2021. Cross-Batch Negative Sampling for Training Two-Tower Recommenders. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '21)*. 1632–1636.
- [41] Ruoxi Wang, Bin Fu, Gang Fu, and Mingliang Wang. 2017. Deep & Cross Network for Ad Click Predictions. In *Proceedings of the ADKDD'17*. Article 12, 7 pages.
- [42] Samuel Williams, Andrew Watterman, and David Patterson. 2009. Roofline: an insightful visual performance model for multicore architectures. *Commun. ACM* 52, 4 (2009), 65–76.
- [43] Jun Xiao, Hao Ye, Xiangnan He, Hanwang Zhang, Fei Wu, and Tat-Seng Chua. 2017. Attentional Factorization Machines: Learning the Weight of Feature Interactions via Attention Networks. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence (IJCAI'17)*. AAAI Press, 3119–3125.
- [44] Xin Xin, Alexandros Karatzoglou, Ioannis Arapakis, and Joemon M. Jose. 2020. Self-Supervised Reinforcement Learning for Recommender Systems. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development*

- in *Information Retrieval (SIGIR '20)*. 931–940.
- [45] Ji Yang, Xinyang Yi, Derek Zhiyuan Cheng, Lichan Hong, Yang Li, Simon Xiaoming Wang, Taibai Xu, and Ed H. Chi. 2020. Mixed Negative Sampling for Learning Two-Tower Neural Networks in Recommendations. In *Companion Proceedings of the Web Conference 2020 (WWW '20)*. 441–447.
 - [46] Zhilin Yang, Zihang Dai, Ruslan Salakhutdinov, and William W. Cohen. 2018. Breaking the Softmax Bottleneck: A High-Rank RNN Language Model. In *International Conference on Learning Representations (ICLR'18)*.
 - [47] Jiaqi Zhai, Yin Lou, and Johannes Gehrke. 2011. ATLAS: A Probabilistic Algorithm for High Dimensional Similarity Search. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data (SIGMOD '11)*. 997–1008.
 - [48] Chang Zhou, Jianxin Ma, Jianwei Zhang, Jingren Zhou, and Hongxia Yang. 2021. Contrastive Learning for Debiased Candidate Generation in Large-Scale Recommender Systems. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 3985–3995.
 - [49] Guorui Zhou, Xiaoqiang Zhu, Chenru Song, Ying Fan, Han Zhu, Xiao Ma, Yanghui Yan, Junqi Jin, Han Li, and Kun Gai. 2018. Deep Interest Network for Click-Through Rate Prediction (*KDD '18*). 10 pages.
 - [50] Han Zhu, Daqing Chang, Ziru Xu, Pengye Zhang, Xiang Li, Jie He, Han Li, Jian Xu, and Kun Gai. 2019. Joint Optimization of Tree-Based Index and Deep Model for Recommender Systems. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Article 357.
 - [51] Han Zhu, Xiang Li, Pengye Zhang, Guozheng Li, Jie He, Han Li, and Kun Gai. 2018. Learning Tree-Based Deep Model for Recommender Systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (London, United Kingdom) (KDD '18)*. 1079–1088.
 - [52] Jingwei Zhuo, Ziru Xu, Wei Dai, Han Zhu, Han Li, Jian Xu, and Kun Gai. 2020. Learning Optimal Tree Models under Beam Search. In *Proceedings of the 37th International Conference on Machine Learning (ICML'20)*.

A HYPERPARAMETER SETTINGS (PUBLIC DATASETS)

For ML-1M, Amazon Beauty, and Amazon Games datasets, we follow the original optimal hyperparams for SASRec reported in [18], specifically two encoder layers ($b = 2$) with one attention head ($h = 1$), and a dropout rate of 0.2 for MovieLens and 0.5 for Amazon Reviews datasets. For ML-20M and Amazon Books, we kept dropout rate identical, grid-searched embedding dimension, number of encoder layers and attention heads for SASRec, and obtained $d = 256$, $b = 4$, $h = 8$ for ML-20M and $d = 64$, $b = 4$, $h = 4$ for Books.

We grid searched MoL parameters (k_u, k_x) in $(2, 2)$, $(4, 4)$, $(8, 8)$ and fixed component-level embedding dimension to 32. For DeepFM experiments, we used the exact same configuration for user- and item- side projections as MoL, and tuned hidden dim and dropout rate additionally. Given the high computational costs of running experiments on ML-20M and Amazon Books datasets (up to 3d for 300 epochs), we shared NeuMF/DeepFM/MoL hparams between those two groups and ML-1M. Detailed configurations can be found in Table 9.

Table 9: Hyperparameters used for public datasets

		ML-1M	ML-20M	Beauty	Games	Books
SASRec	embedding dim d	50	256	50	50	64
	encoder blocks b	2	4	2	2	4
	attention heads h	1	8	1	1	4
	dropout rate	0.2	0.2	0.5	0.5	0.5
Dot Product	temperature	20	20	20	20	20
MLP	hidden layer size	512	512	128	128	256
	dropout rate	0.1	0.1	0.2	0.2	0.2
	Inference FLOPs per (x, u)	50.5K	256.5K	12.6K	12.6K	32.3K
NeuMF	GMF dim	32	32	32	32	32
	MLP hidden dim	256	256	128	128	256
	MLP output dim	64	64	64	64	64
	dropout rate	0.1	0.1	0.2	0.2	0.1
	final MLP hidden dim	256	256	128	128	256
	Inference FLOPs per (x, u)	49.3K	152.3K	24.6K	24.6K	56.3K
FM	hidden dim for output layer	256	256	128	128	256
	prediction layer dropout rate	0.2	0.2	0.3	0.3	0.2
	Inference FLOPs per (x, u)	197.6K	405K	10.7K	10.7K	203.6K
MoL	$(k_u \times k_x \times \text{embedding dim})$	$(8 \times 8 \times 32)$	$(8 \times 8 \times 32)$	$(4 \times 4 \times 32)$	$(4 \times 4 \times 32)$	$(8 \times 8 \times 32)$
	hidden dim for gating MLPs	128	128	128	128	128
	hidden dim for embedding proj. MLPs	512	512	N/A	N/A	512
	item-side embedding proj. MLP dropout rate	0.1	0.1	0.3	0.3	0.1
	gating softmax dropout rate	0.2	0.2	-	-	0.2
	gating input dropout rate	-	-	0.2	0.2	-
	component-level hypersphere embeddings	On	On	Off	Off	On
	τ	20	20	20	20	20
	Inference FLOPs per (x, u)	211.7K	445.3K	12.9K	12.9K	227.6K
	Non-cachable inference FLOPs per (x, u)	32.9K	33.1K	4.4K	4.4K	21.5K